

JS MASTERY MARATHON

WEEK 03 · NOTES & ASSIGNMENT

Loops & Iteration Patterns

Five shapes. Every loop you'll ever write fits one of them.

INSIDE THIS PACK

How to use this

This document has two halves. The notes are what you learned in class — read them before you start the problems. The assignments are 10 problems you'll submit by Friday night.

- | | | |
|----|--|---|
| 01 | The 5 Loop Patterns | Counter, Accumulator, Search, Filter, Transform |
| 02 | for vs while — the 30-second rule | When to use which loop |
| 03 | Live Demo — Password Strength Checker | All 5 stages we built in class |
| 04 | Common Loop Bugs | The 4 mistakes that will bite you |
| 05 | Exercise Pack — 10 Problems | Easy → Medium → Challenge + Bonus |
| 06 | Self-check before submitting | Did you do it right? |
-

THE PROMISE THIS WEEK

By the end of this week, loops stop being scary. Because you'll see that every loop fits one of 5 shapes — and you'll know which is which the moment you read the problem.

PART 0 1

The 5 Loop Patterns

The core mental model. Install this and every loop problem becomes recognizable.

SHAPE	WHEN TO REACH FOR IT
Counter	Repeat N times. "Print the table of 7." "Track 3 attempts left."
Accumulator	Build up ONE result. "Sum of marks." "Total bill." "Longest string."
Search	Find the FIRST match, then stop. "First over-18 user." Uses break.
Filter	Keep ALL matches. "All passing students." "All in-stock items."
Transform	Change every item the same way. "Add GST to every price."

THE BIG IDEA

Every loop you'll ever write fits one of these 5 shapes. When you read a problem, your first question isn't "how do I loop" — it's "which shape is this?"

Pattern 1 · Counter

Use when: You know how many times before you start. Or you need to track how many.

```
// Print 1 to 10
for (let i = 1; i <= 10; i = i + 1) {
  console.log(i);
}
```

Reach for it when you hear: "do X 10 times," "print the table of," "for each year from 2020 to 2025"

Pattern 2 · Accumulator

Use when: You're building up ONE result by combining everything.

```
// Sum of 1 to 100
let total = 0;
for (let i = 1; i <= 100; i = i + 1) {
```

```

    total = total + i;
  }
  console.log(total); // 5050

```

Reach for it when you hear: "total," "sum," "count how many," "average," "longest"

CRITICAL

The accumulator (`total = 0`) goes OUTSIDE the loop. Inside, it resets every iteration — and you'll always end up with the last value, not the total.

Pattern 3 · Search

Use when: You stop the moment you find what you're looking for.

```

// Find first number divisible by 7 and 11
for (let n = 1; n <= 1000; n = n + 1) {
  if (n % 7 === 0 && n % 11 === 0) {
    console.log("Found: " + n);
    break;
  }
}

```

Reach for it when you hear: "first," "find the," "until," "as soon as"

The key word is `break` — it's how you stop early.

Pattern 4 · Filter

Use when: You want to KEEP everything that matches a condition.

```

// All even numbers from 1 to 20
for (let i = 1; i <= 20; i = i + 1) {
  if (i % 2 === 0) {
    console.log(i);
  }
}

```

Reach for it when you hear: "all the X that," "only the," "every item where"

DIFFERENCE FROM SEARCH

Filter keeps going. Search stops at the first match. Same shape — different stopping rule.

Pattern 5 · Transform

Use when: You change every item the same way.

```
// Convert prices from ₹ to $ (assume 83 ₹/$)
for (let i = 1; i <= 5; i = i + 1) {
  let priceInRupees = i * 100;
  let priceInDollars = priceInRupees / 83;
  console.log("₹" + priceInRupees + " = $" + priceInDollars.toFixed(2));
}
```

Reach for it when you hear: "convert every," "apply to all," "add X to each"

PART 02

for vs while

The 30-second rule that ends the "which loop do I use?" question forever.

The rule:

- **for** loop — when you know HOW MANY times before you start
- **while** loop — when you stop based on a CONDITION, not a count

Side by side

```
// Counter — I know I want 10 times → for
for (let i = 1; i <= 10; i = i + 1) {
  console.log("Hello " + i);
}

// Search with unknown end — I stop when guess is correct → while
let guess = 0;
while (guess !== 7) {
  guess = Number(prompt("Guess my number:"));
}
```

REMEMBER

for is for counting. **while** is for waiting. That's it. Forever.

The 30-second test

Ask yourself: "Can I describe this loop without using the word count?"

- Print the table of 7 → count from 1 to 10 → **for**
- Sum 1 to N → count from 1 to N → **for**
- Scan every character → count through indices → **for**
- User tries until they succeed or run out → wait for condition → **while** ✓

WHY THE PASSWORD CHECKER USES WHILE

You don't know how many attempts the loop will run — could be 1, 2, or 3. A `for` loop ASSUMES you'll run all N iterations. A `while` loop ASKS A QUESTION every time before running again. You're waiting for success OR exhaustion — not counting.

PART 03

Live Demo · Password Checker

The 5 stages we built in class. Re-read each one and try to type it from memory before checking.

Stage 1 · Single check, length only

Start simple. No loops yet. Just one check.

```
let password = prompt("Enter a password:");

if (password.length < 8) {
  console.log("Too short. Must be at least 8 characters.");
} else {
  console.log("Length OK.");
}
```

This works for ONE check. But a real password needs 3. Time to add a loop.

Stage 2 · Add digit check (first loop)

You can't just write `if (password has a number)` — JavaScript doesn't work that way. You have to SCAN the password character by character.

```
let password = prompt("Enter a password:");

if (password.length < 8) {
  console.log("Too short.");
}

// Check for a digit – scan every character
let hasDigit = false;
for (let i = 0; i < password.length; i = i + 1) {
  let char = password[i];
  if (char >= "0" && char <= "9") {
    hasDigit = true;
  }
}

if (!hasDigit) {
  console.log("Must contain a number.");
}
```



```
}
```

The trick: `char >= '0' && char <= '9'` checks if a character is a digit. Characters can be compared like numbers — that's it. Don't overthink it.

Stage 3 · Add uppercase check

Same shape as Stage 2 — just a different range of characters.

```
let hasUpper = false;
for (let i = 0; i < password.length; i = i + 1) {
  let char = password[i];
  if (char >= "A" && char <= "Z") {
    hasUpper = true;
  }
}

if (!hasUpper) {
  console.log("Must contain an uppercase letter.");
}
```

NOTICE

Stages 2 and 3 have the EXACT SAME SHAPE. That's not a coincidence — that's the Search pattern. You scan everything looking for a match.

Stage 4 · Wrap in while for 3 attempts

This is the big one. We wrap everything in a while loop and give the user 3 tries. Welcome to nested loops.

```
let attemptsLeft = 3;
let success = false;

while (attemptsLeft > 0 && !success) {
  let password = prompt("Enter a strong password:");
  let isValid = true;

  // Length check
  if (password.length < 8) {
    isValid = false;
    console.log("Too short.");
  }
}
```

```
// Digit check
let hasDigit = false;
for (let i = 0; i < password.length; i = i + 1) {
  if (password[i] >= "0" && password[i] <= "9") {
    hasDigit = true;
  }
}
if (!hasDigit) {
  isValid = false;
  console.log("Must contain a number.");
}

// Uppercase check
let hasUpper = false;
for (let i = 0; i < password.length; i = i + 1) {
  if (password[i] >= "A" && password[i] <= "Z") {
    hasUpper = true;
  }
}
if (!hasUpper) {
  isValid = false;
  console.log("Must contain an uppercase letter.");
}

if (isValid) {
  success = true;
} else {
  attemptsLeft = attemptsLeft - 1;
  console.log("Attempts left: " + attemptsLeft);
}

if (success) {
  console.log("Password set successfully.");
} else {
  console.log("Account locked.");
}
```

WHAT JUST HAPPENED — READ THIS TWICE

There are loops inside loops. The outer `while` controls how many TIMES the user can try. The inner `for` loops scan the password each time. Outer = attempts. Inner = character scan. This is called a nested loop, and you just wrote your first one.

Trace it for yourself:

```
Attempt 1 (while runs)
└─ for: scan chars for digit
└─ for: scan chars for uppercase
→ fail → attemptsLeft = 2
```

```
Attempt 2 (while runs again)
└─ for: scan chars for digit
└─ for: scan chars for uppercase
→ success → exit while
```

Stage 5 · Collect all reasons in an array

Right now, if a password fails 3 rules, we print 3 separate lines. Real apps show all 3 reasons together. Use an array to collect them.

```
let reasons = [];

if (password.length < 8) {
  reasons.push("must be at least 8 characters");
}

// ... digit check, push to reasons ...
// ... uppercase check, push to reasons ...

if (reasons.length === 0) {
  success = true;
} else {
  console.log("Password " + reasons.join(", "));
  attemptsLeft = attemptsLeft - 1;
}
```

LOOK BACK

One while loop. Two for loops. The Search pattern, the Counter pattern, and an array — all in one realistic build. This is what "choosing the right loop pattern" feels like.

PART 04

Common Loop Bugs

The 4 mistakes that will eat your time. Learn to spot them in your own code.

Bug 1 · Off-by-one

The difference between `i <= 10` and `i < 10` is one iteration. Sounds tiny — costs you hours.

```
// Wrong — runs 11 times (0,1,2,...,10)
for (let i = 0; i <= 10; i = i + 1) { ... }

// Right — runs 10 times (0,1,2,...,9)
for (let i = 0; i < 10; i = i + 1) { ... }
```

Rule of thumb: for indices (starting at 0), use `<`. For counts (starting at 1), use `<=`.

Bug 2 · Infinite loop

You forget to update the counter. The loop runs forever and freezes your browser.

```
// Wrong — i never changes, loop never ends
let i = 0;
while (i < 10) {
  console.log(i);
  // forgot: i = i + 1
}
```

Survival tip: if your browser freezes, close the tab. Then check that every `while` loop has something that changes the exit condition.

Bug 3 · Wrong exit condition

You write a condition that can NEVER become false.

```
// Wrong — guess starts at 0 and never changes
let guess = 0;
while (guess !== 7) {
  console.log("Still going...");
  // forgot to ask the user again
}
```

Always ask: CAN this condition ever become false? If not, you have an infinite loop in disguise.

Bug 4 · Accumulator reset

You put `let total = 0` INSIDE the loop instead of outside. Result: total resets every iteration and you always get the last value.

```
// Wrong – total resets every loop, ends at 10
for (let i = 1; i <= 10; i = i + 1) {
  let total = 0;
  total = total + i;
}

// Right – total accumulates correctly to 55
let total = 0;
for (let i = 1; i <= 10; i = i + 1) {
  total = total + i;
}
```

Rule: the accumulator lives OUTSIDE the loop. Always.

PART 05

Exercise Pack

10 problems · 2 Easy · 5 Medium · 3 Challenge · + Bonus

HOW TO TACKLE EACH PROBLEM

- **1. Decompose** — Write out INPUTS, PROCESS, OUTPUT, EDGES in plain English.
- **2. Name the pattern** — Counter? Accumulator? Search? Filter? Transform?
- **3. Pseudocode** — Plain English steps. No JavaScript yet.
- **4. JavaScript** — Only now open the editor. Translate line by line.

Open your editor before writing pseudocode and you'll spend 45 minutes on Problem 9 with still nothing working. **Decompose first. Pseudocode second. JavaScript third.**

PROBLEM 01 ★ EASY

Multiplication Table Printer

Ask the user for a number. Print its multiplication table from 1 to 10.

```
Example: input 7 →  
7 x 1 = 7  
7 x 2 = 14  
...  
7 x 10 = 70
```

EDGES

- User enters 0 (table of zeros — still print it)
- User enters a negative number
- User enters a decimal (e.g., 7.5)

PROBLEM 02 ★ EASY

Count Vowels in a Word

Take a word from the user. Count how many vowels (a, e, i, o, u) it contains. Show the count.

EDGES

- Empty string (count is 0)
- Uppercase vowels ("A" should count)
- Word with no vowels (e.g., "rhythm")

PROBLEM 03 ★★ MEDIUM

Sum of Numbers from 1 to N

Ask the user for a number N. Calculate the sum of all numbers from 1 to N.

Example: $N = 5 \rightarrow 1 + 2 + 3 + 4 + 5 = 15$

EDGES

- $N = 0$ (sum is 0)
- Negative N
- Very large N (e.g., 1,000,000 — should still work fast)

HINT

This is the textbook Accumulator pattern. let total = 0 goes outside the loop. Inside, total = total + i. Trust the pattern.

PROBLEM 04 ★★ MEDIUM

Reverse a String (Without .reverse())

Take a word. Print it reversed. You're not allowed to use `.reverse()`. Use a loop.

Example: "rajdeep" → "peejdar"

EDGES

- Empty string
- Single character ("a" → "a")
- Word with spaces ("code easy" → "ysae edoc")

HINT

Start with an empty result string. Loop the original from LAST character to FIRST. Add each character to the result. Pattern: Accumulator.

PROBLEM 05 ★★ MEDIUM

FizzBuzz

For numbers 1 to 30, print:

- "Fizz" if divisible by 3
- "Buzz" if divisible by 5
- "FizzBuzz" if divisible by BOTH 3 and 5
- The number itself otherwise

EDGES

- Order of checks matters (FizzBuzz before Fizz, or you'll never reach FizzBuzz)
- Numbers like 15, 30 (divisible by both)

PROBLEM 06 ★★ MEDIUM

Find the Largest Number

User enters 5 numbers one by one. After all 5 are entered, print the largest one.

EDGES

- All 5 numbers are the same (largest is still that value)
- All negative numbers (don't assume 0 is the floor)
- User enters a non-number (skip it or warn)

HINT

The Search-with-tracking pattern. Start with `let largest = -Infinity` (so even negative numbers compete). Each loop, compare and update.

PROBLEM 07 ★★ MEDIUM

Print Prime Numbers Up to N

Ask the user for a number N. Print all prime numbers from 2 to N. (A prime is divisible only by 1 and itself.)

EDGES

- N less than 2 (no primes to print)
- N = 2 (only one prime: 2 itself)
- N is large (1000)

HINT

This is a NESTED loop. Outer loop walks 2 to N. For each number, an inner loop checks if anything from 2 to that-number-minus-1 divides it cleanly. If nothing does — it's prime.

PROBLEM 08 ★★★ CHALLENGE

Compound Interest Year-by-Year

A user invests ₹50,000 at 8% annual interest, compounded yearly. Show the balance at the end of each year for 10 years.

```
Year 1: ₹54,000
Year 2: ₹58,320
...
```

EDGES

- Year 0 (initial balance, before any interest)
- Decimal precision — round to 2 decimals
- Interest rate of 0 (balance stays same)

HINT

Transform pattern over time. Start with `balance = 50000`. Each year: `balance = balance + balance × 0.08`. So clean that most students overthink it.

PROBLEM 09 ★★★ **CHALLENGE** · **STAR PROBLEM**

Password Strength Checker with Retry Limit

WHY THIS IS THE STAR

It forces nested loops AND combines 3 loop patterns — Counter (attempts), Search (scan for digit/uppercase), and Conditional Exit. Get this right, and loops stop being scary forever.

Build a password validator. The user has 3 attempts to create a strong password. A strong password must:

- Be at least 8 characters
- Contain at least one digit (0-9)
- Contain at least one uppercase letter (A-Z)

If they enter a weak password, tell them WHY it's weak (which rules failed). After 3 failed attempts, lock them out. On success, say which attempt succeeded.

EDGES

- Empty password
- Password fails multiple rules at once (show ALL reasons, not just the first)
- Success on attempt 1 vs attempt 3
- User exhausts all 3 attempts

HINT

Draw the structure FIRST. Don't open your editor for 10 minutes. You need: an OUTER while loop (attempts), and INSIDE it, FOR loops that scan the password. If you skip the drawing, you'll write 80 lines of tangled if/else and still have bugs.

PROBLEM 10 ★★★ **CHALLENGE**

ATM PIN Block-Out

Simulate an ATM PIN entry. Correct PIN is "4231". User has 3 tries. On each wrong try, tell them how many attempts are left. On the 3rd wrong try, block the card and show: "Card blocked. Contact your branch."

EDGES

- Correct PIN on attempt 1 (don't burn the other 2 attempts)
- Empty input
- User enters non-numeric input

HINT

This is the Counter + Search combo. The `while` exits on EITHER `attemptsLeft = 0` OR correct PIN. Two exit conditions — same shape as the Star problem.

BONUS CHALLENGE

Spotify Playlist Total Duration

A playlist has 7 songs with durations (in seconds): [210, 180, 240, 195, 220, 200, 255]. Calculate:

- Total duration in minutes:seconds format (e.g., "24:40")
- The longest song
- How many songs are over 3 minutes (180 seconds)

Use 3 different loop patterns in one problem. Identify each pattern out loud before you code.

PART 06

Self-check before you submit

Run through this list before you call it done.

- Did I decompose every problem (Inputs, Process, Output, Edges) before coding?
- Did I write pseudocode before JavaScript?
- Did I name the pattern (Counter / Accumulator / Search / Filter / Transform) for each problem?
- Did I handle the EDGES listed in each problem?
- Did I run my code with at least 2 different inputs and verify the output?

IF YOU ONLY DO ONE THING

Do Problem 9. It's the one that makes loops click. Skip the easy ones if you must. Do not skip the Star.

Submission & support

Post your code to the WhatsApp group by Friday night. We solve them together at Logic Lab.

Stuck? Don't give up at Problem 4. The medium ones are the bridge. The Star is the payoff.

See you Friday for Logic Lab. Bring your questions, your bugs, and your wins.

— Rajdeep